

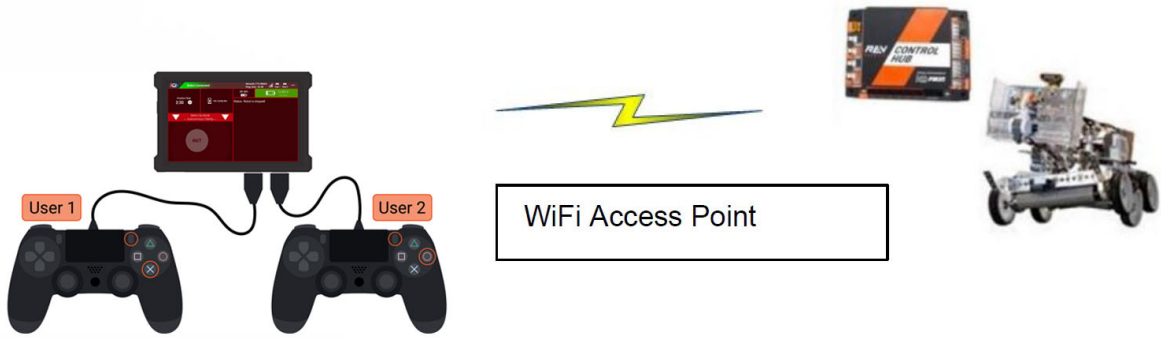
Course 2

注意：先确保你已经clone了当赛季的代码仓库，并下载好了依赖

了解我们的机器人

控制系统

控制系统基于安卓，设备之间使用Wi-Fi通信；其主要分为两部分，操作端与机器端。

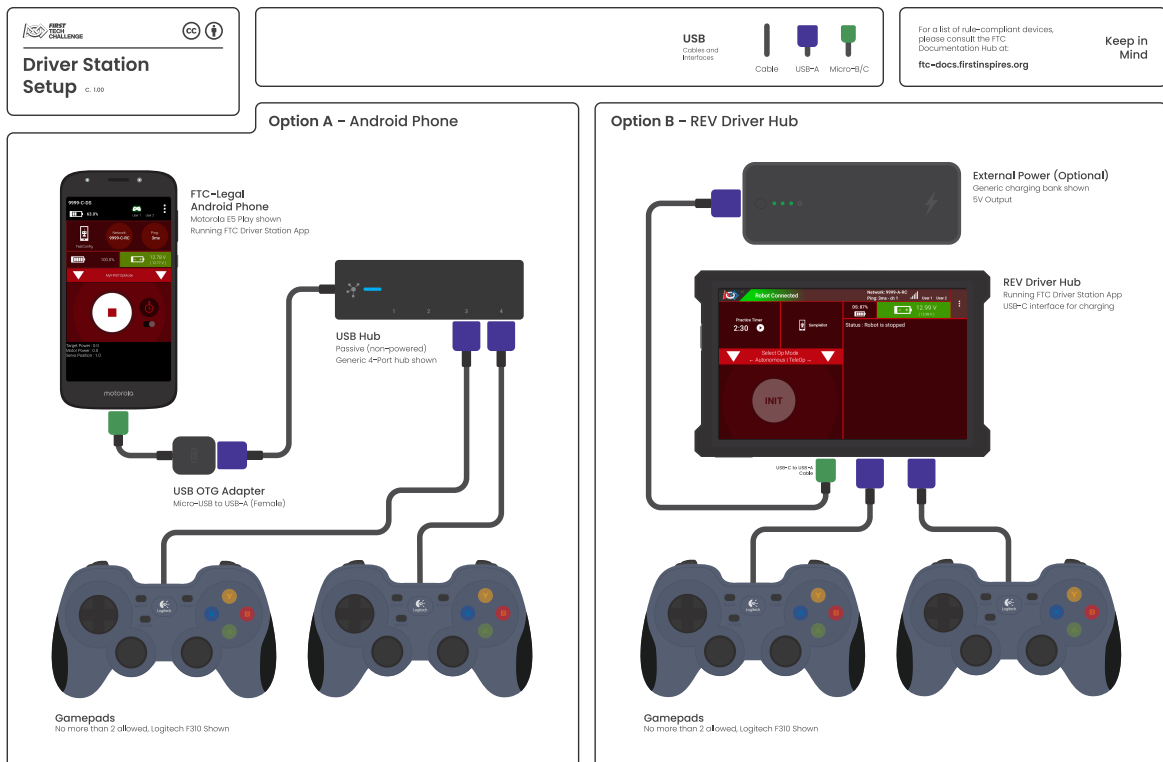


操作端

操作端就像遥控器，它可以远程连接到机器端并给机器人发送指令。

操作端的核心是Driver Hub，其上运行了FtcDriverStation。（Driver Hub是硬件安卓设备，FtcDriverStation是软件安卓应用）

至多两个手柄可以通过USB有线连接到Driver Hub来手动操控机器人。

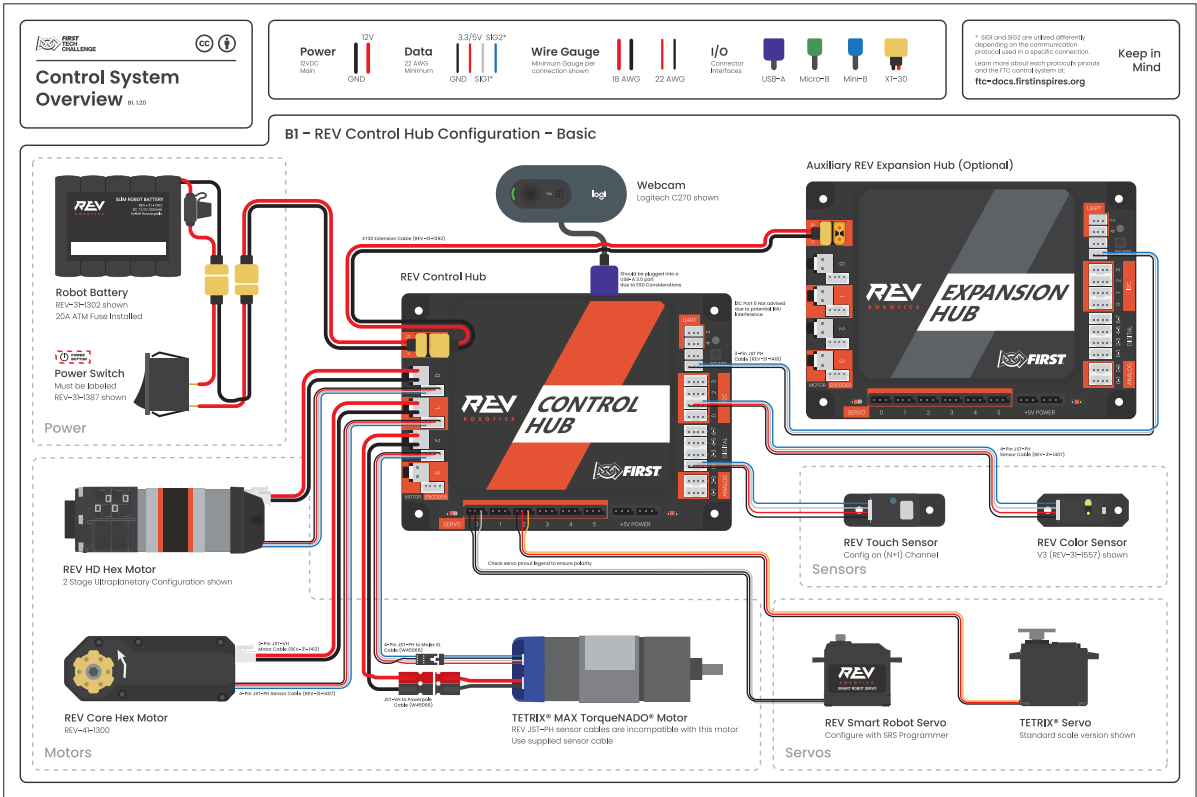


机器端

机器端被固定在机器人上，它就像机器人的大脑一样控制机器人的运动、视觉和“思考”。

机器端的核心是Control Hub，其上运行了FtcRobotController。（Control Hub是硬件安卓设备，FtcRobotController是软件安卓应用）

当特定种类接口不够用时，可以连接至多一个Expansion Hub来获得更多接口。



硬件连接与配置

所有机器人上的配件都要连接到Control Hub或与Control Hub相连的Expansion Hub，并进行相应的配置才能被软件识别并操控。

电池

Control Hub接受12V输入，电池接口为XT-30（黄色、小而扁），REV电池（下左图）即采用XT-30接口。

除REV电池外，实验室还有MATRIX电池（下右图），采用Tamiya接口（透明，长而粗），可通过开关转连接到Control Hub。

两种接口都分公母口，注意不要插反。



接入电池后，Control Hub指示灯会闪烁蓝色，待其变为绿色常亮后便启动完成。若Driver Hub已开机并能连接Control Hub，在Control Hub启动完成时会发出提示音。

电机

我们常用的电机有REV HD Hex Motor和goBILDA电机。二者都配备一个供电接口和一个编码器接口。

要将电机连接到Control Hub，将较粗的12V供电一端接入电机¹，另一端接入Hub左侧四个接口的任意一个；再将较细的编码器线一端接入电机，另一端接入供电线对应的编码器接口（每组供电-编码器接口已用颜色分区）。

舵机

舵机一般没有固定型号，只需将舵机插入Hub下侧接口即可。

大多数舵机连接线都不够长，建议使用舵机延长线。

摄像头

摄像头通常采用USB端口，将其插入Control Hub的USB 3.0端口即可。

Expansion Hub

两个Hub之间使用XT-30接口分享供电，使用RS485接口通信。将供电线和通信线连接好就可以在Expansion Hub上添加更多硬件了。

机器人中的Hello World!

1.连接硬件

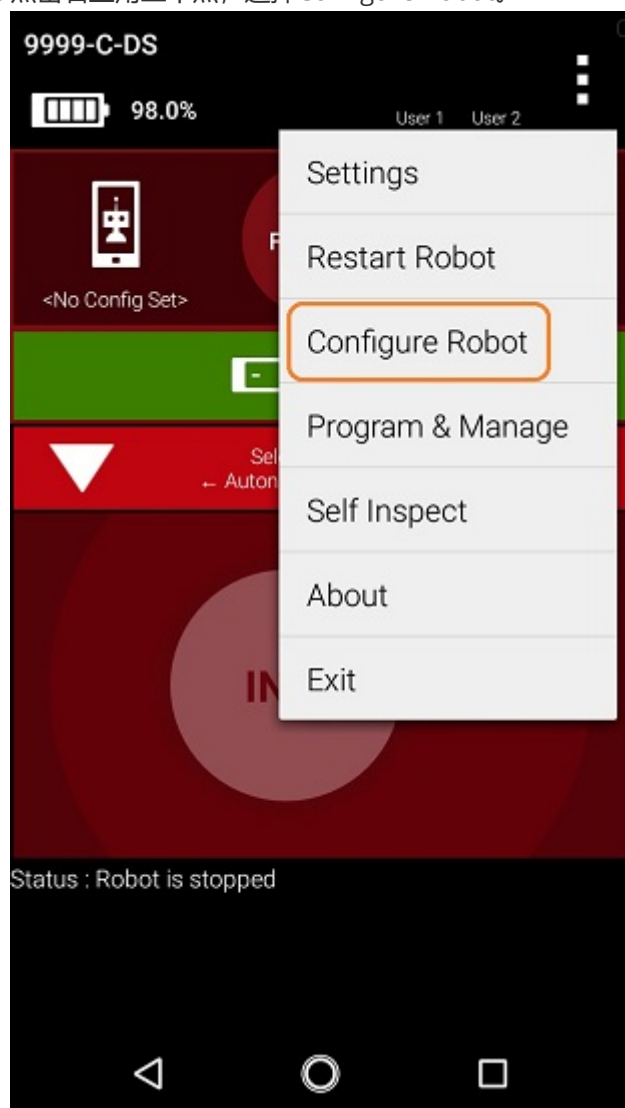
把电机线及其编码器的线插在Motor/Encoder一侧的接口上，并记住你插在了几号口

2.配置

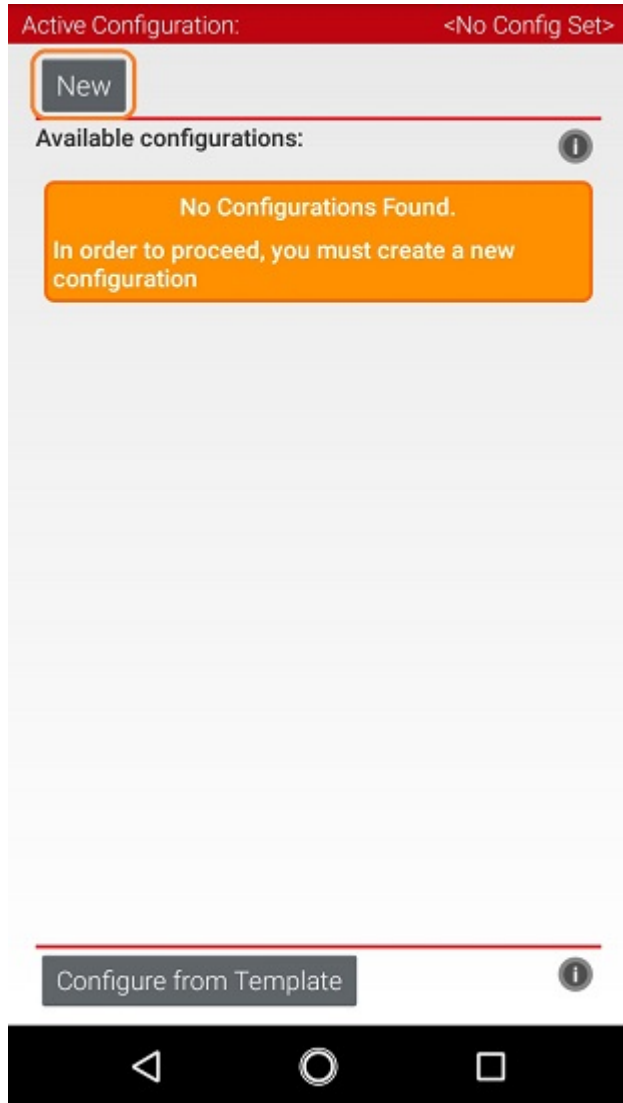
连接好硬件后，我们需要进行配置。我们需要为每个硬件命名，并指定其接入的接口。以电机为例，具体操作如下：

1. 将Driver Hub开机并连接到Control Hub的Wi-Fi，打开FtcDriverStation。

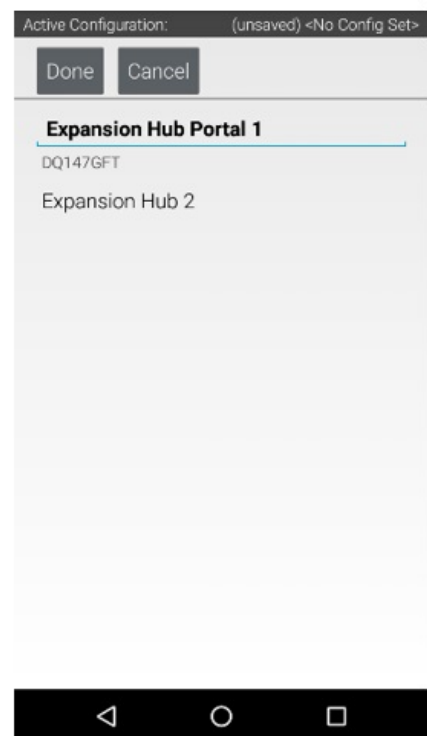
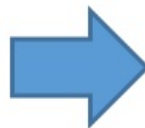
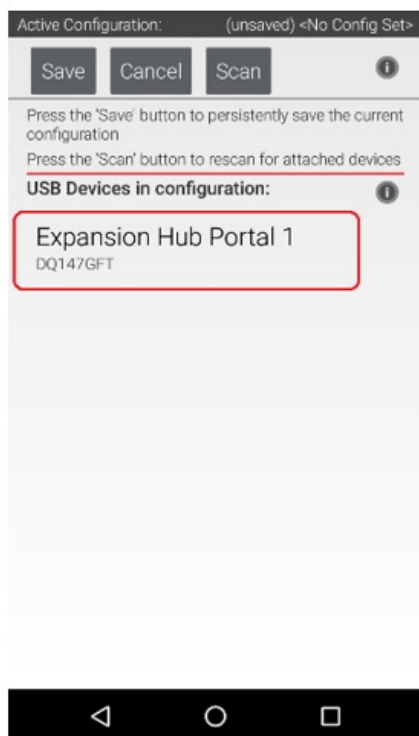
2. 点击右上角三个点，选择Configure Robot。



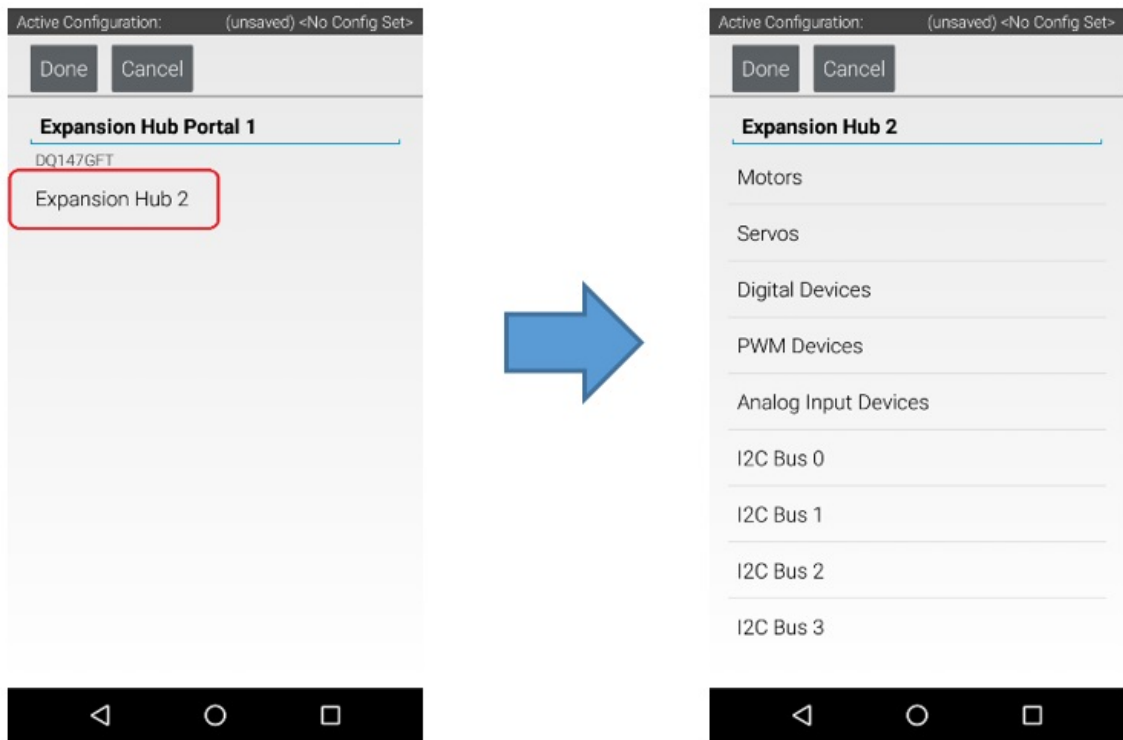
3. 接下来，可以点击左上角New新建配置。若有已创建的配置，点击配置名称下方的Edit来修改它。



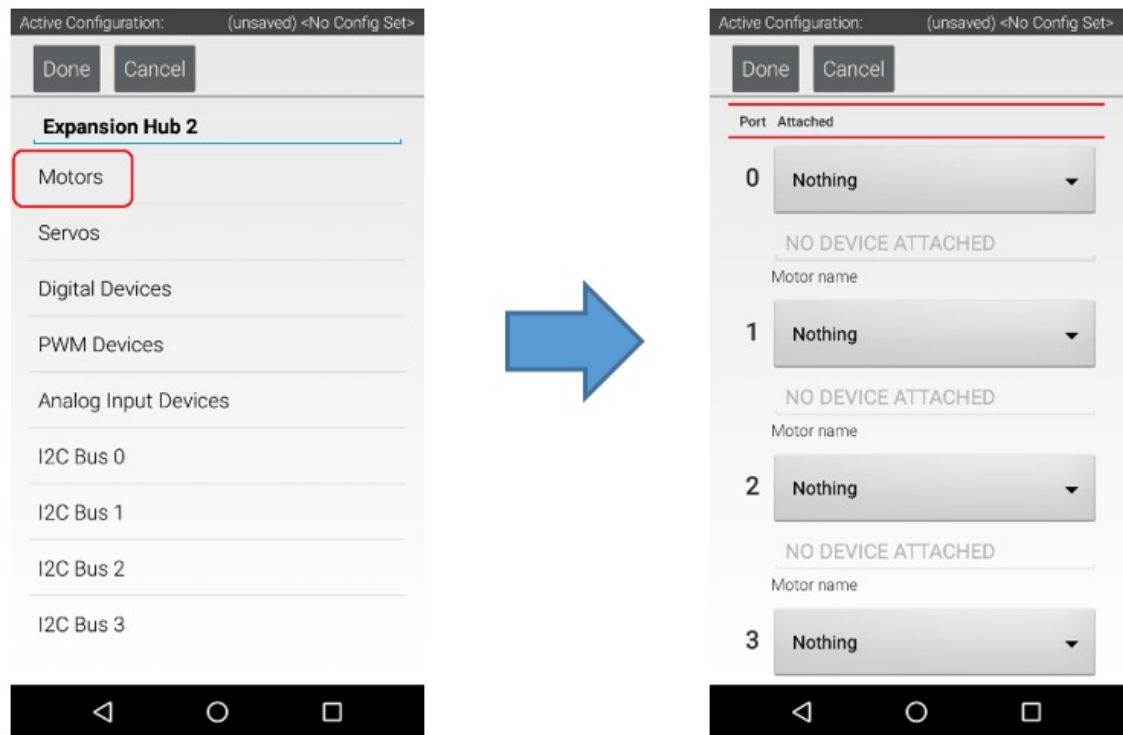
4. 当配置修改界面出现后，系统会自动扫描已有的Hub和摄像头，可以点击Scan重新扫描。点击“Control Hub Portal”来修改连接到Control Hub的硬件配置。



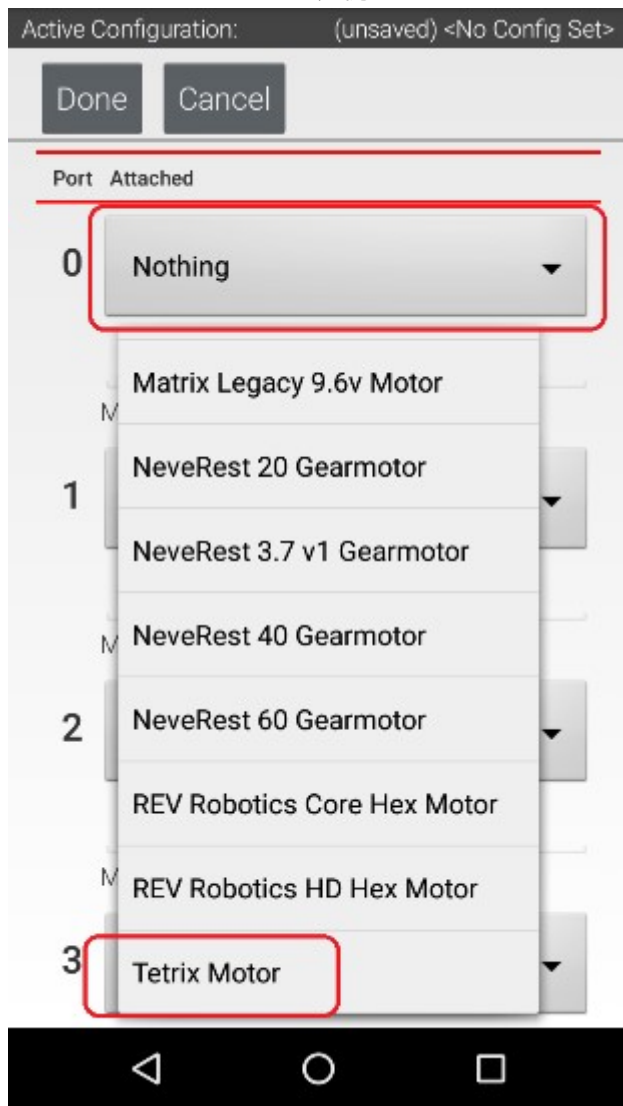
5. 接下来选择你要配置的Hub，若连接了Expansion Hub，这里可以分别设置连接到Control Hub和Expansion Hub的硬件。



6. 选择你要配置的硬件类型。以电机为例，这里我们选择Motors。



7. 点击你所连接的接口编号（这里以0为例）右侧的下拉栏，找到对应的硬件型号。这里以我们常用的Rev HD Hex Motor 20:1为例。



8. 在下方的输入框中输入硬件名称。

Active Configuration: (unsaved) <No Config Set>

Done Cancel

Port	Attached
0	Tetrix Motor
	motorTest Motor name
1	Nothing

motor rest | motor test | motorway

q w e r t y u i o p
a s d f g h j k l
↑ z x c v b n m
? ! , . →

9. 按需重复以上步骤配置其他硬件，当所有硬件配置好后点击Done。

Port	Attached
0	Tetrix Motor motorTest Motor name
1	Nothing NO DEVICE ATTACHED Motor name
2	Nothing NO DEVICE ATTACHED Motor name
3	Nothing

10. 点击Save来保存当前配置，在输入框中输入该配置的名字。

11. 使用Activate激活配置，若有多个配置，可以使用每个配置对应的Activate按钮快速切换。按返回按钮退出配置界面。

至此，配置完成，可以使用代码来操控机器。

3.代码编写

操作模式是用来自定义机器人的行为的一系列代码集合，其本质是继承了

`com.qualcomm.robotcore.eventloop.opmode.OpMode` 类的一个类。

在编写操作模式时，通常继承 `com.qualcomm.robotcore.eventloop.opmode.LinearOpMode` 类，因为它提供更多方法。

示例：MyOpMode.java

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;

@TeleOp
public class MyOpMode extends LinearOpMode {

    @Override
```

```

public void runOpMode() {
    DcMotor motor = hardwareMap.get(DcMotor.class, "Motor");

    waitForStart();

    while (opModeIsActive()) {
        motor.setPower(-gamepad1.left_stick_y);
        sleep(10);
    }

    motor.setPower(0);
}
}

```

接下来我们将逐行分解这个操作模式。

- **包**：所有队伍自己编写的代码都应放在 `org.firstinspires.ftc.teamcode` 包² 中，以使 Robot Controller 可以读取到队伍代码。
- **@TeleOp**：@TeleOp 是一个注解²，用来标识这个操作模式是手动模式。同样的，还有 @Autonomous 作为自动模式的标识。
若一个操作模式没有被@TeleOp或@Autonomous标记，那么这个操作模式将不可用。
- **runOpMode方法**：一个操作模式主要的代码都集中在这里。这里的代码在你按下 Driver Hub 上的 INIT 键之后就会执行。这个方法可以分为几部分：
 - **初始化**：按下 INIT 键之后执行一次。调用 `waitForStart()` 之前。
 - **开始运行**：按下三角形 START 键之后执行一次。调用 `waitForStart()` 之后，`while (opModeIsActive())` 循环之前。本例中这个阶段不需要操作，故省略。
 - **运行循环**：按下 START 键之后循环执行。`while (opModeIsActive())` 循环。
 - **结束清理**：按下方形 STOP 键之后执行一次。`while (opModeIsActive())` 循环之后。

初始化

```
DcMotor motor = hardwareMap.get(DcMotor.class, "Motor");
```

这里定义了一个 motor 变量，类型为 `DcMotor`，意为直流电机。

`hardwareMap` 是 `OpMode` 类的一个属性，它存储有当前激活的硬件配置的映射。使用 `hardwareMap.get(Class<?>, String)` 来根据硬件名称获取某个特定类型的硬件。如这里的 `hardwareMap.get(DcMotor.class, "Motor")`，意思就是从硬件配置中获取名字为“Motor”的直流电机。方法返回一个硬件对象。

运行循环

这里的代码会在按下 START 键之后重复运行，因此保证这段代码的高效性是十分关键的。

```
motor.setPower(-gamepad1.left_stick_y);
```

使用 `DcMotor#setPower(double)` 方法来控制电机旋转速度，参数为 `[-1.0, 1.0]` 的 `double`，0 对应不转，1 对应正向最大速度，-1 对应反向最大速度，中间小数等比例换算成电机速度。

`gamepad1` 是 `OpMode` 类的一个属性，它存有此刻手柄1的按键状态。`gamepad1.left_stick_y` 代表手柄1左摇杆竖直方向位置，类型为 `double`，范围`[-1.0,1.0]`。使用前加符号是因为读取时摇杆向前是负值而向后是正值，为了符合直觉做出这样的调整。

```
sleep(10);
```

`sleep(long)` 是 `LinearOpMode` 的一个方法，可以让操作模式暂时休眠（不运行任何代码），从而节省计算资源，或是完成特定目标。参数为一个 `long` 值，代表要休眠的毫秒数。（1秒=1000毫秒）

结束清理

```
motor.setPower(0);
```

将电机速度设为0，机器停止运行。

至此，代码就讲解完了。

4.烧录

代码需要烧录到Control Hub上才能被运行。有两种烧录方式：有线与无线。

有线

将一根USB-A到USB-C数据线插入Control Hub的USB-C口，另一端接入电脑，Android Studio右上角就应出现Control Hub v1.0字样。点击右侧绿色三角形运行按钮（或是灰色返回样式重新运行按钮），即可将代码烧录到Control Hub。待Control Hub重启完成，便可以启动你的操作模式，开始操作了。

无线

先找到电脑上Android SDK的安装位置，然后将 `<你的SDK路径>\platform-tools\` 加入系统PATH环境变量中³。电脑连接Control Hub的Wi-Fi⁴，在命令行中输入 `adb connect 192.168.43.1`，连接到Control Hub的调试接口。这时Android Studio应该能识别到Control Hub，然后就可以用上面一样的方法进行烧录了。

5.手柄相关

在FtcDriverStation启动后，首次将手柄插入Driver Hub时，需要按下手柄上的Start+A或Start+B键来向Driver Hub说明这是手柄1还是手柄2。如果Driver Hub识别成功，Driver Hub右上角会出现一个小小的手柄图标。在FtcDriverStation没有重启的情况下，拔出再插入手柄会自动识别，无需按键。

现在，你就可以尝试编写代码，用手柄操控机器人了！

硬件的基本控制

直流电机

电机可以无限制地旋转，并读取对应的编码器位置，**通常用于底盘，以及其他需要快速旋转的装置**

```
DcMotor leftMotor = hardwareMap.get(DcMotor.class, "Left Motor");
DcMotor rightMotor = hardwareMap.get(DcMotor.class, "Right Motor");

DcMotor elevatorMotor = hardware.get(DcMotor.class, "Elevator Motor");
DcMotor intakeMotor = hardware.get(DcMotor.class, "Intake Motor");
```

在 `DcMotor` 实例化之后，你可以设置一些变量来影响直流电机的运行方式。第一个变量是方向：

```
leftMotor.setDirection(DcMotor.Direction.REVERSE);
rightMotor.setDirection(DcMotor.Direction.FORWARD);
```

改变电机方向的作用与预期完全一致，即改变方向。如果在正转模式下对电机施加 1 的功率，它将朝一个方向转动。如果处于反转状态，1 的功率将使其向另一个方向旋转。如果将电机轴朝向自己，则正转是逆时针方向（NeveRest 电机除外）。

接下来，有两种零功率行为可以调整：

```
leftMotor.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.BRAKE);
rightMotor.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.FLOAT);
```

更改该变量会影响直流电机在功率为 0 时的行为。如果电机正在运动，`BRAKE` 会使电机减速（如果电机尚未运动，则不会使电机保持位置），而 `FLOAT` 会使电机滑行到停止位置，让摩擦力完成所有工作。

最后，有四种不同的运行模式可用于直流电机：

```
leftMotor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);
rightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);

elevatorMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);
intakeMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
```

需要注意的是，只要正确插入编码器，就可以在上述任何模式下读取编码器值。这些模式只是改变电机对这些编码器值的反应方式。

作为电机，做主要的方法是 `setPower(double power)`，输入范围是 $[-1,1]$ ，控制电机的选择功率。

```
leftMotor.setPower(1);
rightMotor.setPower(1);
```

编码器

- 编码器

编码器是指记录（通常）围绕轴的旋转运动的设备。编码器分为绝对编码器和相对编码器。绝对编码器将准确报告轴与其绝对“零点”相比的角度。相对编码器将报告 [shaft](#) 自开始跟踪以来（例如，自主启动时）旋转了多远。相对编码器通过正交信号输出，而绝对编码器通常通过模拟信号或 i2c 输出。编码器用于帮助找到机器人或其部分机械结构的位置。

虽然所有 FTC 合规电机都内置有相对正交编码器，但必须单独接线，且并非必须使用。只要使用正交通信协议，就 可以使用外部编码器并将其插入编码器端口。

访问编码器需要调用 `DcMotor` 对象上的一个方法 `getCurrentPosition()`，该方法可返回插入端口的编码器的当前位置。该数字在 `opmode` 开始时可以是任意的，除非使用了 `STOP_AND_RESET_ENCODERS` 或对 `Expansion Hub` 进行了电源循环，否则不会重置为 0。

舵机

相较于电机，它旋转有范围，但是可以精确控制他的选择角度，所以用于**精确控制**

```
Servo servo = hardwareMap.get(Servo.class, "servo");
```

在实例化 `Servo` 后，可以调用两个主要函数：`setPosition()` 和 `getPosition()`

```
releaseServo.setPosition(0.75);
```

`setPosition(double position)` 设置舵机的位置。SDK 将使用舵机的电位器的内置控制循环，将舵机驱动到该位置并保持该位置。`setPosition(double position)` 接收介于 0 和 1 之间的 double，其中 0 是舵机的旋转下限，1 是舵机的旋转上限。介于两者之间的是正比例，所以 0.5 是中间值，0.75 是 3/4 上限值，等等。

`getPosition()` 返回的不是servo的**当前位置**，而是当前**目标位置**。如果servo的当前目标位置的变量存储得当，就不需要使用此函数。

特别注意：该方法返回的实际上是**上一次setPosition时传入的参数**，这也就意味着，在程序刚开始时，我们无法知道实际上舵机的位置，`getPosition()`输出为0

*连续旋转舵机

```
CRServo servo = hardwareMap.get(CRServo.class, "servo");
```

CRServo 有一个主要方法：`setPower()`。其工作原理与 `DcMotor` 的 `setPower()` 非常相似，即传入 0 会使其停止，传入 1 会使其全速前进，传入 -1 会使其全速后退，以及介于两者之间的所有情况。

所以CRServo通常不用于精确控制，而是作为小型电机使用

```
servo.setPower(0.75);
```

其他传感器

传感器术语表

具体用法可以在代码仓库中的 `org.firstinspires.ftc.robotcontroller.external.samples` 包中查询

也可以在JavaDoc中查看更多用法

一些有用的网站

[FTC官方文档](#)

[Game Manual 0](#)⁵

[FTC Javadoc](#)

[FtcRobotController GitHub 仓库](#)

1. goBILDA电机可能需要转接头或焊接。 ↩

2. 关于这些Java概念，在这里不多做赘述，可以把他们当成固定的格式来记忆。当然，如果你想要更进一步学习Java而不是单纯比赛，强烈建议上网自学这些概念。 [↵](#) [↵](#)
3. 修改PATH环境变量的方法详见： <https://jingyan.baidu.com/article/ff42efa9542467819e2202d3.html> [↵](#)
4. 注意，连接Control Hub的Wi-Fi之后电脑便不能通过Wi-Fi上网。请自行准备有线连接或在需要上网时与Control Hub断开连接，但重新连接后电脑有时不能识别Control Hub，需要重新使用adb连接到Hub。 [↵](#)
5. Game Manual 0 简体中文翻译由 上海市川沙中学 FTC 19656 Absolute Zero 绝对零度 的队员们贡献，在此鸣谢他们的努力与付出。 [↵](#)